DATA PRODUCT CASE STUDY · STELLARALGO

One source of truth for a multi-tenant product

How I brought every client's data into one layer that reporting, AI and our CRM could trust.

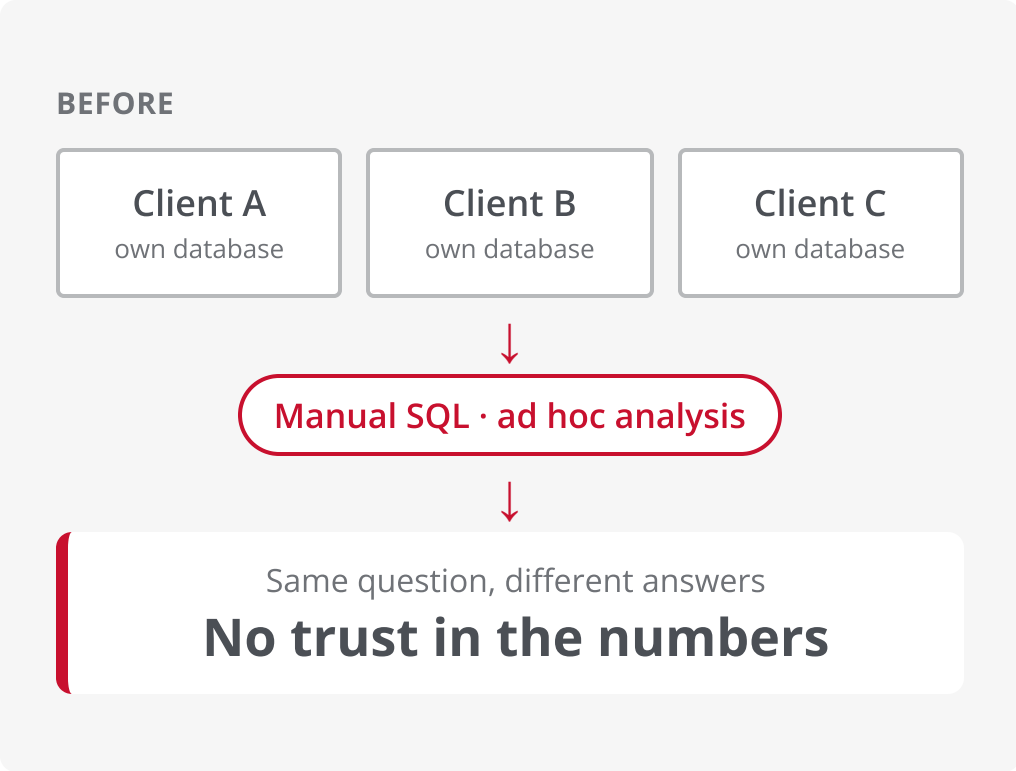
Cody Chandler
Data and Business Analyst interview · The City of Calgary

I'll answer them in order

1. What business problem was I solving?
2. What is the data product?
3. How did I approach it?
4. What challenges came up?
5. What am I most proud of?
6. What would I improve with more time?

What business problem was I solving?

The problem: we had no single source of truth, so reporting and analysis couldn't stay consistent or reliable as we grew.



! What was happening

Each client's data lived in its own database. Reports were up to **7 days old**, and nobody owned the numbers.

a Why it mattered

Every cross-client question meant manual SQL and ad hoc analysis, and teams still ended up with different numbers.

b Users and interested parties

Customer Success · Product · Clients · Analysts

What is the data product?

a One governed, tested layer for every client's data, refreshed four times a day.



GOVERNED THROUGHOUT · Role-based access · Data contract · Freshness monitoring · Tested every run

The data is transformed once, and every team and tool uses the same tables.

What is the data product?

b How people use it.

BI dashboards

Self-serve reporting on the tested gold tables.

Leadership, Product, Customer Success

AI assistant

Ask in plain English. The semantic layer picks the model with the right grain.

“How is ticket revenue trending?”

In the CRM

Customer Success and account managers see the data inside the CRM they already use.

Includes ML results

All three read the same tables, so the numbers match.

What is the data product?

c The value it delivered.

FRESH

~~Weekly, by hand~~

4× a day

Refreshes run automatically, on a set schedule.

CONSISTENT

~~Different answers~~

One number

Every report, dashboard and tool uses the same definitions.

OBSERVABLE

~~Found by clients~~

Monitored

We check freshness, tests, pipeline errors and cost on every run.

SELF-SERVE

~~Engineering tickets~~

Self-serve

Customer Success answers routine questions without engineering.

Teams now work from the same numbers, and they trust them.

How did I approach it?

I started with what people needed, then fixed the data at the source.

1 Requirements

The questions each team couldn't answer, and the numbers they disputed.

2 Strategy

Fix the data in one place, not in every report.

3 Plan

Medallion layers, a data contract, one grain per table.

4 Build

dbt models in Snowflake, orchestrated by Airflow.

5 Review & iterate

Tests and monitoring catch issues, and we fix them at the source.

a Sourcing and preparing the data

Product analytics, customer data and client config.

Land
Raw, personal data masked

→

Map
Client by client, into one model

→

Test
On every run, before use

b Methods and tools

BUILT IN **dbt**	STORED IN **Snowflake**	ORCHESTRATED BY **Airflow on AWS**
VERSIONED IN **GitHub**	DOCUMENTED IN **dbt docs, Confluence**	SERVED THROUGH **Semantic layer, QuickSight, Claude**

What challenges came up?

What got in the way, and how I handled it.

Type	Challenge	How I solved it
Technical	Every client system had its own shape.	One model all clients map into.
Analytical	The same metric meant different things to different teams.	One definition, one documented grain per table.
People	Nobody owned freshness or accuracy.	A data contract I wrote: owners, targets and incident tiers.

Each fix went into the shared layer, so every report picked it up.

What am I most proud of?

The AI gives the same answer as the dashboards.

QUESTION

"How is ticket revenue trending?"

→

SEMANTIC LAYER: ROUTE TO THE RIGHT GRAIN

Property by day	One row per property, per day ✓
Campaign	One row per campaign
Other models	Each with one documented grain

→

ANSWER

Same number as the dashboard

SQL shown, within the user's role

The governance work is what made self-serve possible.

What would I improve with more time?

What I'd do next.

Measure from day one

Track hours and escalations before launch, so I could show the impact in numbers.

Take it to clients

Offer the internal AI assistant to clients, on the same layer.

Cover more domains

Add semantic models for more of the business, so the AI can answer more.

At the City I'd do the same: find where the numbers disagree, agree on definitions with the owners, and make the data easy to use.

Thank you.

I can't show you StellarAlgo's data, so I rebuilt the same approach on City of Calgary open data. Let's look at it live.

LIVE DEMO

codychandler.ca/data-demos/calgary-transit